Comparing Forecasters for Serverless Invocation Provisioning

1st Mojtaba Jafari
Department of Computer Engineering
Amirkabir University of Technology
Tehran, Iran
mojtaba.jafari@aut.ac.ir

2nd Amir Kalbasi
Department of Computer Engineering
Amirkabir University of Technology
Tehran, Iran
kalbasi@aut.ac.ir

***Abstract*—Proactive capacity provisioning mitigates serverless cold starts by pre-warming containers, yet practical autoscaling is constrained by the memory cost of idle instances. Evaluating forecasters under an operational capacity model reveals that conventional statistical error metrics obscure essential provisioning trade-offs. Because container instantiation follows a discrete step function, continuous forecasters induce severe idle memory inflation; enforcing an operational half-invocation cutoff resolves this artifact, collapsing median idle capacity from thousands of container-minutes down to approximately 100 (0.5 for LSTM, 1.5 for Chronos, and 112.5 for ETS) while uncovering true capacity shortfalls previously masked by continuous non-zero predictions. Benchmarking seven core forecasters spanning recency heuristics, classical smoothing, and deep/foundation models (alongside reactive baselines) across 80 Azure Functions workloads shows that lightweight recency heuristics (Previous Hour, Seasonal Naive, and ETS) achieve the lowest point-forecast errors, rivaling or outperforming deep LSTMs and foundation models (Chronos-Bolt). Mitigating capacity shortfalls (an operational proxy for cold starts) is governed by headroom rather than model complexity: tracking upper arrival quantiles (Trailing P95 or Chronos P90) slashes median shortfall from 75.0 to 18.5 container-minutes. Finally, archetype-specific tuning over brief windows fails to beat uniform policies across temporal shifts, while reactive keep-alive baselines underscore the latency penalty of hourly forecasting.**

***Index Terms*—serverless computing, workload forecasting, cold start, provisioning**

I. Introduction

Serverless computing abstracts infrastructure management through fine-grained, event-driven execution, yet its elasticity remains constrained by cold-start latency. When an incoming invocation finds no warm instance, it incurs a substantial initialization delay. While keeping idle containers warm eliminates this penalty, it consumes costly memory resources. Serverless capacity provisioning thus contends with an inherent dilemma: minimizing user-facing cold starts while curtailing wasteful idle memory footprint.

Predictive provisioning addresses this tension by forecasting invocation demand to pre-warm containers proactively. However, a fundamental disconnect exists between standard time-series evaluation and practical cloud capacity management. Conventional statistical metrics, such as sMAPE or MSE, treat errors symmetrically. In production environments, forecast errors carry strictly asymmetric consequences: under-prediction yields container shortfalls and cold starts, whereas over-prediction leaves instances idle. Crucially, container allocation operates as a discrete step function, meaning that even infinitesimal continuous forecasts can trigger full container allocations. Consequently, models optimized purely for statistical loss can produce severely distorted provisioning decisions.

Compounding this challenge is the extreme heterogeneity of real-world serverless workloads. Production traces, such as Microsoft Azure Functions [1], reveal heavily skewed behaviors across sparse dormancy, diurnal patterns, and unpredictable bursts. Prior studies typically embed forecasting models inside complex autoscaling controllers, entangling pure predictive accuracy with ad-hoc scheduling heuristics. It remains an open question how diverse forecasters perform when cleanly isolated under an explicit capacity provisioning model.

In this work, we conduct a systematic, capacity-driven empirical comparison of invocation forecasters across four representative workload archetypes (sparse, daily, bursty, and other) sampled from the Azure trace. Across 80 functions, we evaluate seven core strategies spanning simple training-free heuristics (previous hour, last-value, trailing percentiles, seasonal naive), classical smoothing (seasonal Holt–Winters / ETS), deep recurrent networks (LSTM), and zero-shot foundation models (Chronos-Bolt [2]), alongside operational reference baselines. Because production traces record minute-level aggregates rather than individual request arrival times or kernel boot latencies, we evaluate point accuracy alongside container shortfall (an operational proxy for cold-start risk) and memory-weighted idle costs over 60-minute horizons.

Our empirical investigation reveals three central findings. First, continuous forecasters require an explicit cutoff for fractional predictions (< 0.5); without it, discrete container rounding acts as an accidental 24/7 keep-alive, artificially inflating idle capacity into thousands of container-minutes. Second, lightweight heuristics and Holt–Winters match or surpass deep learning and foundation models in point accuracy, while cold-start shortfall is governed by quantile headroom rather than model complexity. Third, per-archetype model selection fails to outperform simple baselines under weekend distribution shifts, and hourly proactive horizons inherently lag behind fine-grained reactive keep-alive mechanisms.

xxxx-..-....-....-x/xx/$xx.00 ©2026IEEE

II. Related Work

A. Workload Characterization and Keep-Alive Strategies

Cold-start mitigations broadly encompass container caching, runtime optimizations, and predictive resource provisioning [3]. In their seminal characterization of Microsoft Azure Functions, Shahrad et al. revealed extreme workload skewness: most functions are invoked infrequently, inter-arrival times span multiple orders of magnitude, and memory allocation is heavily concentrated [1]. Multi-tenant traces analyzed by Joosen et al. corroborated this skewness, noting that invocation forecasting remains an open challenge [4]. While hybrid histogram policies tune idle timeouts [1] and Lambda-Warmer adapts keep-alive windows [5], keep-alive mechanisms remain inherently reactive, leaving platforms vulnerable to cold starts when bursts follow idle periods and motivating proactive workload forecasting.

B. Predictive Autoscaling and Proactive Schedulers

To anticipate arrival surges before cold starts occur, recent research embeds predictive modeling directly into serverless autoscaling loops. Rather than evaluating forecasters in isolation, contemporary frameworks bind predictions to custom scaling controllers. For instance, AAPA [6] classifies invocation patterns on Kubernetes to assign policies with uncertainty bounds, while Daraghmeh et al. deploy classifiers for scaling decisions [7]. Hybrid and control-theoretic frameworks pair ARIMA with random forests [8] or Fourier forecasting with model predictive control [9]. Deep models have predicted cold starts for provisioned concurrency [10], and Transformers have targeted pre-warming on Azure traces [11]. However, such end-to-end evaluations entangle forecasting accuracy with scheduling heuristics; in contrast, we isolate forecasters under a unified capacity protocol to evaluate their direct provisioning gaps.

C. Time-Series Forecasters: From Heuristics to Foundation Models

Classical baselines, including seasonal naive repetition and Holt–Winters smoothing, remain ubiquitous benchmarks. Recently, time-series foundation models have emerged as zero-shot alternatives: Chronos tokenizes continuous series for transformer forecasting [2], Chronos-Bolt emits multi-horizon quantiles in one pass, and Chronos-2 adds multivariate support [12], alongside concurrent models such as TimesFM [13] and Moirai [14]. While foundation models eliminate per-function training, their capacity-aware efficiency remains uncharacterized. We evaluate Chronos-Bolt against classical baselines on error and quantify the idle capacity its quantiles expend to suppress shortfalls.

III. Setup

We evaluate the public Microsoft Azure Functions dataset [1], recording per-minute invocation counts over 14 days (missing days treated as 1,440 zeros). We classify functions into four archetypes based on days 1–10 profiling: *sparse* functions exhibit zero invocations in at least 90% of recorded minutes (representing 54,502 of the 68,394 fleet functions); *daily* functions exhibit strong diurnal periodicity, identified by a lag-1440 autocorrelation exceeding 0.30; *bursty* functions capture volatile, non-periodic arrivals where the coefficient of variation reaches at least 2.05 (the 75th percentile among non-sparse functions); and *other* functions comprise the remaining non-periodic workloads. We sample 20 eligible functions per archetype (seed 42), requiring at least one invocation in days 1–10 and 11–14 (80 functions total). All evaluation code, function identifiers, and trained models are publicly available at https://github.com/Mojtaba0jafari/serverless-invocation-forecasting.

Forecasts are generated for a 60-minute lookahead horizon and re-issued hourly across days 11–14, producing 96 non-overlapping evaluation windows per function. Days 1–10 serve as historical training and profiling data, while days 11–14 constitute the held-out test period. In addition, weekend days 13–14 are analyzed separately to assess whether model rankings established on weekdays (days 11–12) persist under temporal distribution shifts. For context input, seasonal Holt–Winters, Chronos, and the LSTM observe a 24-hour historical window (1,440 minutes), whereas recency heuristics (last-value and trailing percentiles) condition strictly on immediate observations.

The primary evaluation compares seven core forecasting strategies: 1) *Seasonal Naive* repeats the corresponding 60 minutes from the previous day when lag-1440 autocorrelation exceeds 0.30, defaulting to repeating the preceding hour otherwise; 2) *Previous Hour* directly projects the preceding 60 minutes of observed demand; 3) *Last Value* projects the single latest observed minute uniformly across the hour; 4) *Trailing P95* projects the 95th percentile of the preceding 10 minutes; 5) *ETS (Holt–Winters)* employs additive exponential smoothing with daily seasonality, optimizing smoothing parameters per function via MSE grid search on days 1–10; 6) *Chronos* utilizes the median point forecast from pretrained `amazon/chronos-bolt-small` [2] zero-shot; and 7) *LSTM* employs a shared single-layer architecture (32 units, linear output) trained on sliding windows from days 1–8 with MAE loss, early-stopped on days 9–10, with inputs scaled by context mean.

Several reference baselines provide additional comparative context: unconditional *Always-Previous-Day* copies; intermittent-demand models (Croston and TSB [15], smoothing $\alpha = 0.1$); high-quantile variants including Chronos (80th and 90th percentiles) and an LSTM trained under pinball loss ($\tau = 0.9$); and multiplicative safety margins ($1.25\times$, $1.5\times$, $2.0\times$). In addition, a *reactive keep-alive* serves as an operational reference benchmark: updated per-minute, it maintains warm container capacity matching the peak usage of the preceding 10 minutes, representing an idealized reactive controller rather than a fixed 60-minute lookahead forecast.

To translate continuous invocation forecasts into discrete resource allocations, our capacity model assumes a nominal concurrency of 10 invocations per container-minute. Crucially, raw continuous predictions below 0.5 invocations are

clamped to zero; without this threshold, infinitesimal non-zero predictions (e.g., 0.01) round up to a full container, artificially distorting idle capacity. For an actual invocation count y_t and forecast $\hat{y}_t$ (clipped at zero), required containers $r_t = \lceil y_t/10 \rceil$ and prepared containers $p_t = \lceil \hat{y}_t/10 \rceil$ determine the operational metrics:

$$S = \sum_t \max(0,\, \lceil y_t/10 \rceil - \lceil \hat{y}_t/10 \rceil)\,, \tag{1}$$

$$O = \sum_t \max(0,\, \lceil \hat{y}_t/10 \rceil - \lceil y_t/10 \rceil)\,. \tag{2}$$

Here, S measures *container shortfall* (in container-minutes), serving as a trace-driven operational proxy for cold starts since the Azure dataset aggregates invocations into 1-minute bins and lacks sub-minute arrival timestamps or kernel boot latencies. Conversely, O measures *idle capacity* (extra container-minutes). To quantify the economic footprint of over-provisioning, we calculate memory-weighted idle overhead ($O \times$ memory) using each function's application-level median allocated memory from days 1–10 (sample median 144 MB, range 90–518 MB). Because container provisioning follows discrete steps, numerical deviations that stay within the same container boundary incur neither penalty nor waste. A third metric, *new containers*, defined as $\sum_t \max(0,\, r_t - \max(p_t, r_{t-1}))$, accounts for container reuse: an unabsorbed surge spanning multiple consecutive minutes counts once per instantiated container rather than accumulating per minute.

For statistical comparison, forecast accuracy is measured using symmetric Mean Absolute Percentage Error (sMAPE) per minute:

$$\mathrm{sMAPE}_t = 100 \cdot \frac{2|y_t - \hat{y}_t|}{|y_t| + |\hat{y}_t|}, \tag{3}$$

where minutes with zero actual and predicted demand contribute zero. Because a few high-volume functions dominate fleet aggregates, we summarize per-function distributions using class and fleet medians. Statistical significance is evaluated via 95% percentile bootstrap confidence intervals (and paired differences) computed from 2,000 within-class resamples.

IV. Evaluation

A. Scoring cutoff

Initial unthresholded scoring revealed severe idle capacity inflation: ETS, LSTM, and Chronos incurred median extra capacities of 2,101.5, 3,640.5, and 3,281.0 container-minutes. Because container allocation rounds up ($\lceil \hat{y}_t/10 \rceil$), infinitesimal predictions (< 0.5, present on 88% of idle minutes in Chronos and 50% in ETS) acted as accidental 24/7 keep-alives. Enforcing a half-invocation cutoff ($\hat{y}_t < 0.5 \implies 0$) resolves this artifact, collapsing idle medians to 112.5, 0.5, and 1.5 container-minutes while exposing genuine capacity shortfalls (shifting medians to 52.5, 303.5, and 253.5). Sweeping $\tau \in [0.10, 1.00]$ reveals a sharp elbow: over 98% of idle savings occur by $\tau = 0.25$ (dropping Chronos idle capacity to 3.5), and $\tau = 0.50$ robustly balances idle footprint and shortfall across models. All subsequent analyses enforce this 0.5 cutoff.

B. Error

Table I summarizes point-forecast accuracy across test days 11–14. Simple heuristics attain the lowest fleet median sMAPE: Previous Hour copy achieves 4.7%, followed by Seasonal Naive (7.1%) and ETS (8.2%). The paired difference between ETS and Seasonal Naive is negligible (0.0 points, 95% CI: $[-0.0, 0.7]$). Deep and foundation architectures incur higher aggregate error: Chronos-Bolt yields 13.5%, LSTM 17.0%, Last Value 19.2%, and Trailing P95 24.7%. Bootstrap intervals (2.7%–15.3% for Seasonal Naive) confirm that differences among the top three fall within estimation noise. Archetype splits highlight specialized competencies: ETS leads on daily workloads (2.1% vs. 3.9% for Seasonal Naive); Chronos excels on irregular other functions (7.8% vs. 12.9%); and Chronos (34.8%) and LSTM (34.6%) outperform Seasonal Naive (41.0%) on bursty traces. On sparse functions, Chronos and LSTM tie at 0.7% by predicting near zero. Seasonal Naive's autocorrelation gate yields negligible fleet benefit: copying yesterday produces 9.6% sMAPE versus 4.7% for the previous hour, confirming that local recency dominates diurnal periodicity at minute granularity. Fig. 1 illustrates representative daily and sparse forecasts.

C. Shortfall and idle capacity

As shown in Table II, every model incurs shortfall across most functions (66% for Trailing P95 to 90% for Previous Hour). Seasonal Naive achieves a balanced operating point, with median shortfall of 75.0 container-minutes (95% CI: $[29, 169]$) and 80.5 extra container-minutes. In contrast, conservative point forecasters—Chronos median (253.5 short, 1.5 extra) and LSTM (303.5 short, 0.5 extra)—minimize idle memory at the cost of high shortfall. Conversely, Trailing P95 trades idle capacity for latency protection, cutting median shortfall to 18.5 container-minutes (paired difference -35.5, CI: $[-108.5, -15.0]$) while inflating extra capacity to 729.0 ($+426.5$, CI: $[214, 1{,}228]$), scaling memory from 15.3k to 135.4k MB-minutes but halving new containers from 25.5 to 12.5. Chronos at its 90th percentile achieves virtually the same trade-off (18.5 short, 754.5 extra, 8.5 new containers), while ETS occupies an intermediate point (52.5 short, 112.5 extra).

Fig. 2 depicts the Pareto frontier when systematic headroom is added across pooled fleet demand. Operating at upper quantiles (trailing percentiles or Chronos quantiles) curtails shortfall slightly more cost-effectively than scalar multipliers ($1.25\times, 1.5\times$). However, no amount of proactive headroom drives pooled shortfall below $\sim$14% of required capacity, bounded by unpredicted spikes in bursty workloads. An LSTM trained with pinball loss ($\tau = 0.9$) approaches this frontier with higher idle waste (20.5 short, 1,205.5 extra). Outperforming all proactive curves, the reactive 10-minute keep-alive achieves a median shortfall of 6.0 container-minutes (1.6% pooled shortfall at 43% extra), demonstrating the latency penalty of hourly forecasting.

D. Class-specific choices and the test days

Table III investigates whether specialized per-archetype forecaster selection outperforms fleet-wide uniform policies. We evaluate two tuning policies calibrated on weekday observations (days 11–12): a *shortfall-first* policy (ranking by shortfall, then idle capacity) and a *balanced* policy (minimizing combined short and extra container-minutes). Days 13–14 span a weekend where fleet invocations drop steeply (1.23M and 0.99M vs. 3.41M and 3.06M on weekdays, mirroring troughs on days 6–7), reducing error medians fleet-wide. The shortfall-first policy selects ETS for sparse functions and Trailing P95 for all others, matching Trailing P95 on days 13–14. The balanced policy selects LSTM for sparse, ETS for daily, Chronos for bursty, and Trailing P95 for other workloads. Although this hybrid records 5 short and 0 extra container-minutes on days 13–14 (vs. 12 and 15 for Seasonal Naive), paired differences are statistically indistinguishable from zero (0 short, 95% CI: $[0, 0]$; 0 extra, CI: $[-2.0, 0.0]$). Thus, brief calibration windows offer insufficient predictive stability to justify archetype-specific selection across temporal regime shifts.

E. Capacity and weighting

We assess the robustness of our nominal 10-invocation container capacity via two sensitivity checks. First, given a sample median execution duration of 326 ms, a sequential container could theoretically execute ∼184 invocations per minute. Sizing capacity dynamically to each function's average duration (floored at 1) yields medians of 32.5 short and 38.0 extra for Seasonal Naive, 5.0 and 246.0 for Trailing P95, 29.0 and 34.5 for ETS, 69.0 and 0.0 for Chronos, and 88.5 and 0.0 for LSTM—strictly preserving relative model rankings. Second, reweighting functions by true fleet prevalence (where sparse workloads represent ∼80% of traces) yields medians of 17 short and 17 extra for Seasonal Naive, 15 and 129 for Trailing P95, 16 and 12 for ETS, and 24 and 0 for Chronos. Hence, our conclusions remain robust under duration-scaled capacity and fleet-proportional weighting.

F. Limits

Several limitations contextualize these findings. First, evaluation relies on trace-driven simulation across 80 Azure functions rather than live telemetry; shortfall (S) and instantiations serve as capacity proxies rather than kernel measurements. Second, containers are modeled per function rather than across multi-function applications. Third, static 60-minute forecast horizons favor persistence baselines over continuous sliding re-forecasts. Fourth, we benchmarked a fixed 10-minute keep-alive rather than adaptive histogram policies [1]. Finally, deep models were evaluated out-of-the-box (shared LSTM and zero-shot Chronos-Bolt) without extensive hyperparameter tuning.

TABLE I
MEDIAN SMAPE, SHORT CONTAINER-MINUTES, AND EXTRA CONTAINER-MINUTES ON DAYS 11–14.

Model	Metric	Sparse	Daily	Bursty	Other	All
Seasonal naive	sMAPE (%)	0.9	3.9	41.0	12.9	7.1
	Short	13.0	45.0	718.0	150.0	75.0
	Extra	13.0	43.0	719.0	150.5	80.5
Previous hour	sMAPE (%)	1.1	3.3	41.0	12.9	4.7
	Short	15.5	37.0	718.0	150.0	66.0
	Extra	15.5	43.0	719.0	150.5	69.5
Last value	sMAPE (%)	0.9	37.8	48.2	11.9	19.2
	Short	17.0	1,150	763.5	99.5	410.0
	Extra	0.0	31.0	593.5	93.5	63.5
Trailing P95	sMAPE (%)	3.8	134.4	74.6	12.9	24.7
	Short	15.0	0.0	143.0	0.0	18.5
	Extra	87.5	4,386	2,183	98.0	729.0
ETS	sMAPE (%)	0.9	2.1	55.1	13.7	8.2
	Short	13.0	45.0	615.5	47.0	52.5
	Extra	9.5	8.0	969.5	166.5	112.5
LSTM	sMAPE (%)	0.7	39.9	34.6	12.0	17.0
	Short	19.0	1,150	924.0	0.0	303.5
	Extra	0.0	0.0	125.5	142.5	0.5
Chronos	sMAPE (%)	0.7	34.3	34.8	7.8	13.5
	Short	19.0	1,088	996.5	0.0	253.5
	Extra	0.0	0.0	48.0	144.0	1.5

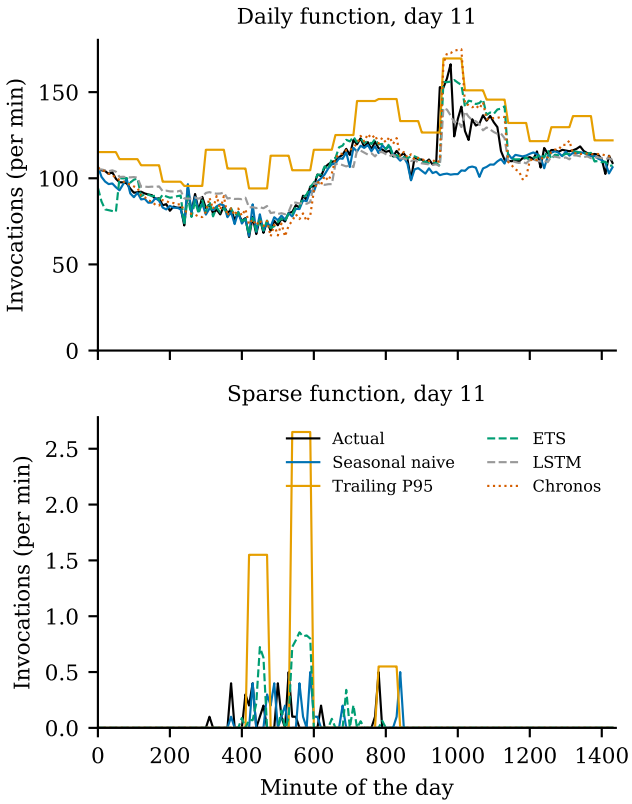


Fig. 1. Day 11 for one daily and one sparse function, in 10-minute means. The other forecasters follow the same shape or stay near zero.

V. CONCLUSION

Proactive provisioning seeks to mitigate serverless cold starts without inflating idle memory overhead. Evaluating diverse forecasting architectures across the Azure Functions benchmark reveals four key operational findings. First, translating continuous forecasts into integer containers induces step-function distortion; enforcing an operational half-invocation cutoff ($\hat{y}_t < 0.5 \implies 0$) resolves this artifact

TABLE II

ALL 80 FUNCTIONS, DAYS 11–14. MEDIANS. NEW IS NEW CONTAINERS, MB IS THOUSANDS OF EXTRA MEGABYTE-MINUTES, AND SHORT% IS THE SHARE OF FUNCTIONS WITH ANY SHORTFALL. THE STAR MARKS A REACTIVE REFERENCE THAT READS ACTUAL COUNTS.

Model	sMAPE	Short	Extra	New	MB	Short%
Seasonal naive	7.1	75.0	80.5	25.5	15.3	88
Previous hour	4.7	66.0	69.5	25.0	12.6	90
Last value	19.2	410.0	63.5	141.5	12.4	88
Trailing P95	24.7	18.5	729.0	12.5	135.4	66
ETS	8.2	52.5	112.5	21.0	17.0	79
LSTM	17.0	303.5	0.5	141.5	0.1	84
LSTM, tau 0.9	35.3	20.5	1,206	6.0	194.6	68
Chronos	13.5	253.5	1.5	127.0	0.4	85
Chronos q90	22.9	18.5	754.5	8.5	111.2	68
TSB	17.3	224.0	10.0	111.0	1.4	84
Croston	32.5	233.0	10.0	111.0	1.4	81
Keep-alive 10 min*	–	6.0	1,038	6.0	157.2	69

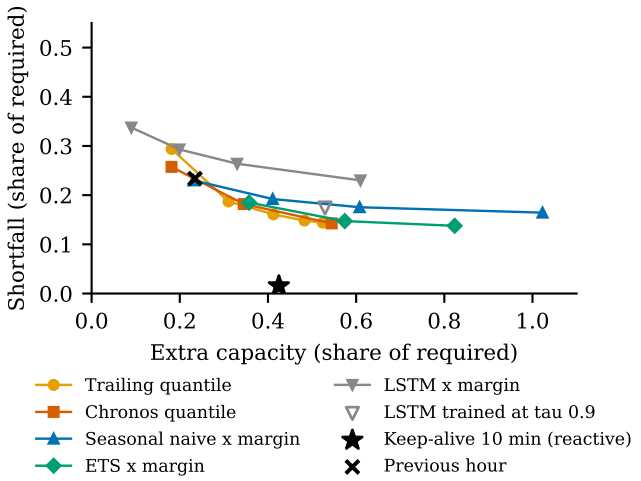


Fig. 2. Pooled short and extra container-minutes as a share of the required container-minutes on days 11–14, with headroom added to each forecaster. Closer to the lower left is better.

by preventing models from acting as unintended 24/7 keep-alives, reducing idle waste to ≈ 100 container-minutes while uncovering genuine capacity trade-offs. Second, simple recency heuristics—Previous Hour, Seasonal Naive, and ETS—demonstrate the lowest fleet-wide error, matching or outperforming complex deep LSTMs and zero-shot Chronos-Bolt. Third, cold-start protection is bought through headroom rather than architectural complexity: upper arrival quantiles (Trailing P95 and Chronos P90) slash median shortfall from 75.0 to 18.5 container-minutes, though unpredicted bursts impose a $\sim$14% lower bound on proactive provisioning that reactive keep-alive baselines navigate more flexibly. Finally, per-archetype model selection over brief calibration windows does not beat uniform policies across temporal shifts. Future work will explore sub-hourly rolling re-forecasting and kernel-level validation in production clusters.

TABLE III

MEDIANS FOR ALL FUNCTIONS ON DAYS 11–12 AND ON THE WEEKEND DAYS 13–14. THE TWO PICKS CHOOSE A FORECASTER PER CLASS ON DAYS 11–12.

	Days 11–12			Days 13–14		
Policy	sMAPE	Short	Extra	sMAPE	Short	Extra
Seasonal naive	8.4	48.5	59.0	2.7	12.0	15.0
Trailing P95	32.6	13.0	694.5	16.2	0.5	120.5
ETS	9.4	30.5	55.5	7.8	8.5	39.5
LSTM	21.2	271.5	0.5	12.3	70.5	0.0
Chronos	15.7	199.5	1.0	7.6	47.0	0.0
Pick: shortfall first	26.4	12.5	694.5	12.7	0.0	120.0
Pick: short plus extra	7.8	37.5	2.0	1.6	5.0	0.0

REFERENCES

[1] M. Shahrad, R. Fonseca, I. Goiri, G. Chaudhry, P. Batum, J. Cooke, E. Laureano, C. Tresness, M. Russinovich, and R. Bianchini, "Serverless in the wild: Characterizing and optimizing the serverless workload at a large cloud provider," in *2020 USENIX Annual Technical Conference (USENIX ATC 20)*. USENIX Association, Jul. 2020, pp. 205–218.

[2] A. F. Ansari, L. Stella, C. Turkmen, X. Zhang, P. Mercado *et al.*, "Chronos: Learning the language of time series," *Transactions on Machine Learning Research*, 2024.

[3] M. Golec, G. K. Walia, M. Kumar, F. Cuadrado, S. S. Gill, and S. Uhlig, "Cold start latency in serverless computing: A systematic review, taxonomy, and future directions," *ACM Computing Surveys*, vol. 57, no. 3, pp. 1–36, 2025.

[4] A. Joosen, A. Hassan, M. Asenov, R. Singh, L. Darlow, J. Wang, and A. Barker, "How does it function? characterizing long-term trends in production serverless workloads," in *Proceedings of the 2023 ACM Symposium on Cloud Computing*, 2023, pp. 443–458.

[5] Y. Singh and R. Shrestha, "Lambda-warmer: A measurement-driven adaptive framework for cost-aware cold-start mitigation in AWS lambda," *Future Generation Computer Systems*, vol. 185, p. 108681, 2026.

[6] G. Zhang, S. Vippagunta, R. Nandagopal, S. Raman, J. Xu, M. Pfeiffer *et al.*, "AAPA: An archetype-aware predictive autoscaler with uncertainty quantification for serverless workloads on Kubernetes," *arXiv preprint arXiv:2507.05653*, 2025.

[7] M. Daraghmeh, Y. Jararweh, and A. Agarwal, "Leveraging machine learning and feature engineering for optimal data-driven scaling decision in serverless computing," *Simulation Modelling Practice and Theory*, vol. 140, p. 103090, 2025.

[8] L. Shiekhani, H. Wang, W. Shi, J. Liu, Y. Qiu, C. Gu, and W. Ding, "The hybrid model: Prediction-based scheduling and efficient resource management in a serverless environment," *Applied Sciences*, vol. 15, no. 14, p. 7632, 2025.

[9] C. Nguyen, M. Bhuyan, and E. Elmroth, "Taming cold starts: Proactive serverless scheduling with model predictive control," *arXiv preprint arXiv:2508.07640*, 2025.

[10] N. S. Kumar and S. S. Samy, "Cold start prediction and provisioning optimization in serverless computing using deep learning," *Concurrency and Computation: Practice and Experience*, vol. 37, no. 4–5, 2025.

[11] A. S. F. M. Mouen, J. L. Zeutouo, and V. K. Tchendji, "Transformer-based model for cold start mitigation in FaaS architecture," *arXiv preprint arXiv:2504.11338*, 2025.

[12] A. F. Ansari, O. Shchur, J. Küken, A. Auer, B. Han, P. Mercado *et al.*, "Chronos-2: From univariate to universal forecasting," *arXiv preprint arXiv:2510.15821*, 2025.

[13] A. Das, W. Kong, R. Sen, and Y. Zhou, "A decoder-only foundation model for time-series forecasting," in *Proceedings of the 41st International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 235, 2024, pp. 10 148–10 167.

[14] G. Woo, C. Liu, A. Kumar, C. Xiong, S. Savarese, and D. Sahoo, "Unified training of universal time series forecasting transformers," in *Proceedings of the 41st International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 235, 2024, pp. 53 140–53 164.

[15] R. H. Teunter, A. A. Syntetos, and M. Z. Babai, "Intermittent demand: Linking forecasting to inventory obsolescence," *European Journal of Operational Research*, vol. 214, no. 3, pp. 606–615, 2011.